LOUP-GAROU

Application web de jeu social multijoueur

RAPPORT DE PROJET

Membres de l'équipe

PIERRE Jean Samuel
COULIBALY Bourama
CHAUVET Darren

13 avril 2026

Table des matières

Introduction **2**

1 Inscription, Connexion, Lobby et réception des informations **3**
1.1 Inscription 3
1.2 Connexion 4
1.3 Lobby 5
1.4 Réception des informations en jeu 6

2 Design CSS global, gestion Git et diagrammes **8**
2.1 Charte graphique et thème visuel 8
2.2 Composants CSS notables 9
2.3 Gestion du dépôt Git et diagrammes 9
2.3.1 Diagramme de cas d'utilisation 10

3 Page d'aide, mécanique des parties, cartes et pixel art **11**
3.1 Page d'aide 11
3.2 Design pixel art des cartes 12
3.3 Priorité d'appel des rôles — Ordre des phases de nuit 13
3.4 Effets et interactions entre les cartes 14
3.5 Conditions de victoire 15
3.6 Système d'auto-avance et compte à rebours 15

Conclusion **17**

Introduction

Ce projet est une implémentation web multijoueur du jeu *Loup-Garou de Thiercelieux*. C'est un jeu de déduction sociale où les joueurs sont secrètement assignés à des rôles et doivent travailler en équipe pour éliminer le camp adverse.

Technologies utilisées :

- **Backend** — PHP (traitement des actions, gestion de l'état, verrous fichiers)
- **Frontend** — JavaScript ES6 (modules), HTML5, CSS3
- **Stockage** — Fichiers JSON avec verrouillage concurrent (`flock`)
- **Communication** — Polling HTTP toutes les 2 secondes (pas de WebSocket)

La partie s'organise en phases successives :

$$\texttt{attente} \to \texttt{distribution} \to \texttt{nuit} \to \texttt{jour} \to \texttt{vote} \to \cdots \to \texttt{fin}$$

Le rapport est organisé selon les contributions de chaque membre :

Partie 1 PIERRE Jean Samuel — Authentification, Lobby, Réception des informations

Partie 2 COULIBALY Bourama — CSS, gestion Git et diagrammes

Partie 3 CHAUVET Darren — Page d'aide, mécanique de jeu, cartes pixel art

Chapitre 1

Inscription, Connexion, Lobby et réception des informations

Contribution de ***PIERRE Jean Samuel***

1.1 Inscription

Fichiers : `register.html` / `php/register.php`

La page d'inscription présente un formulaire demandant trois champs : pseudo, adresse e-mail et mot de passe.

Traitement côté serveur (`register.php`) :

1. Les données sont reçues en JSON via `php://input`.
2. Les champs sont validés (présence obligatoire des trois valeurs).
3. Un verrou exclusif (`flock LOCK_EX`) est posé sur `users.lock` afin d'éviter les conditions de concurrence lors de l'écriture.
4. Le fichier `data/users.json` est lu. Si le pseudo ou l'e-mail existe déjà, une erreur est renvoyée.
5. Le mot de passe est haché avec `password_hash(..., PASSWORD_BCRYPT)`.
6. Le nouvel utilisateur est ajouté au fichier JSON.
7. La session PHP est initialisée (`$_SESSION['pseudo']` et `['mail']`).

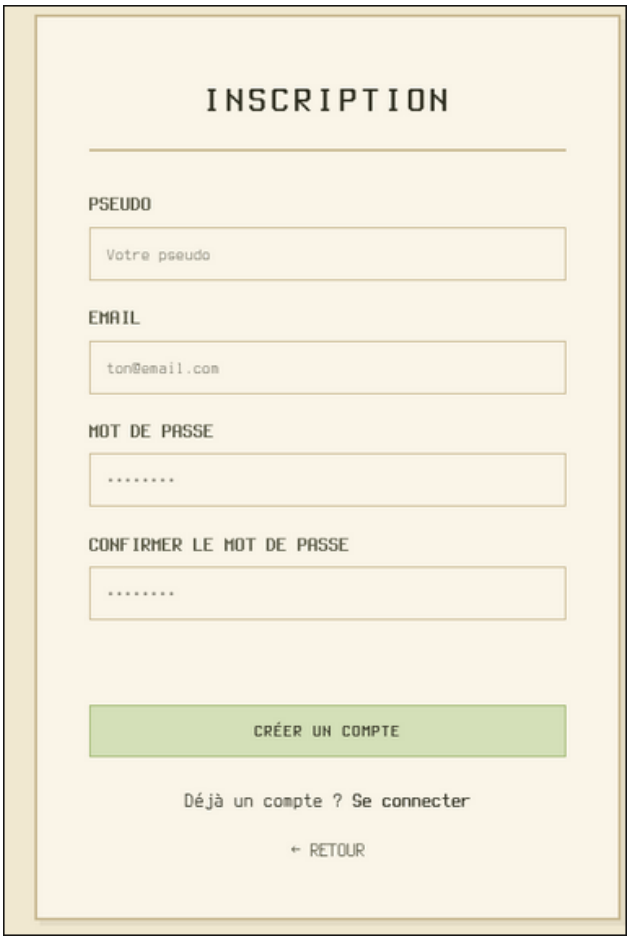

FIGURE 1.1 – Page d'inscription

1.2 Connexion

Fichiers : `login.html` / `php/login.php`

Le formulaire de connexion demande l'e-mail et le mot de passe.

Traitement côté serveur (`login.php`) :

1. Lecture du fichier `users.json` avec un verrou partagé (`LOCK_SH`) — plusieurs utilisateurs peuvent se connecter simultanément.
2. Recherche du pseudo associé à l'e-mail fourni.
3. Vérification du mot de passe avec `password_verify()`.
4. En cas de succès, la session est ouverte (`$_SESSION['pseudo']`).
5. Toutes les pages protégées redirigent vers `login.html` si la session est absente.

FIGURE 1.2 – Page de connexion

1.3 Lobby

Fichiers : `index.php` / `php/creategame.php` / `php/parties.php`

Une fois connecté, le joueur arrive sur le lobby (`index.php`).

Création d'une partie (`creategame.php`) :

- Le joueur choisit le nombre maximum de joueurs (4–20), si la partie est publique ou privée, et les rôles spéciaux à activer (Voyante, Sorcière, Chasseur, Cupidon, Petite Fille).
- Un code à 6 caractères alphanumériques est généré aléatoirement, en excluant les caractères ambigus (O/0, I/1).
- La structure JSON initiale de la partie est créée dans `data/partie_XXXXXX.json` avec la phase `"attente"`.
- La partie est référencée dans `data/parties.json` (index global).
- Un jeton CSRF est validé avant chaque action d'écriture.

FIGURE 1.3 – Lobby — Créer une partie

Rejoindre une partie (`parties.php`) :

- Le joueur peut entrer un code directement ou choisir dans la liste des parties publiques en phase `"attente"`.
- `parties.php` lit l'index, supprime les entrées fantômes (fichiers manquants) et nettoie les fichiers de parties terminées depuis plus de 60 secondes.

FIGURE 1.4 – Lobby — Liste des parties publiques

1.4 Réception des informations en jeu

Fichier : `php/etat.php`

`etat.php` est l'endpoint de polling : le client JS l'interroge toutes les 2 secondes pour obtenir l'état courant de la partie. Le système de filtrage garantit que chaque joueur ne reçoit que les informations auxquelles son rôle lui donne droit.

Information	Destinataires
`phase, tour, liste joueurs, morts`	Tous
`potionVie, potionMort, victime loups`	Sorcière uniquement
`peutTirer`	Chasseur uniquement
`votesLoups`	Loups-Garous uniquement
`resultatVoyante, cibleVoyante`	Voyante uniquement
`resultatEspionnage`	Petite Fille uniquement
`nbVotes, votesJour, candidatsRevote`	Tous (phase vote)
`Tous les rôles`	Tous (phase fin)

TABLE 1.1 – Filtrage de l'état par rôle

Gestion de la concurrence : `etat.php` pose un verrou partagé (`LOCK_SH`) sur le fichier `.lock` de la partie, ce qui lui permet de coexister avec d'autres lectures simultanées tout en attendant qu'`action.php` libère son verrou exclusif (`LOCK_EX`).

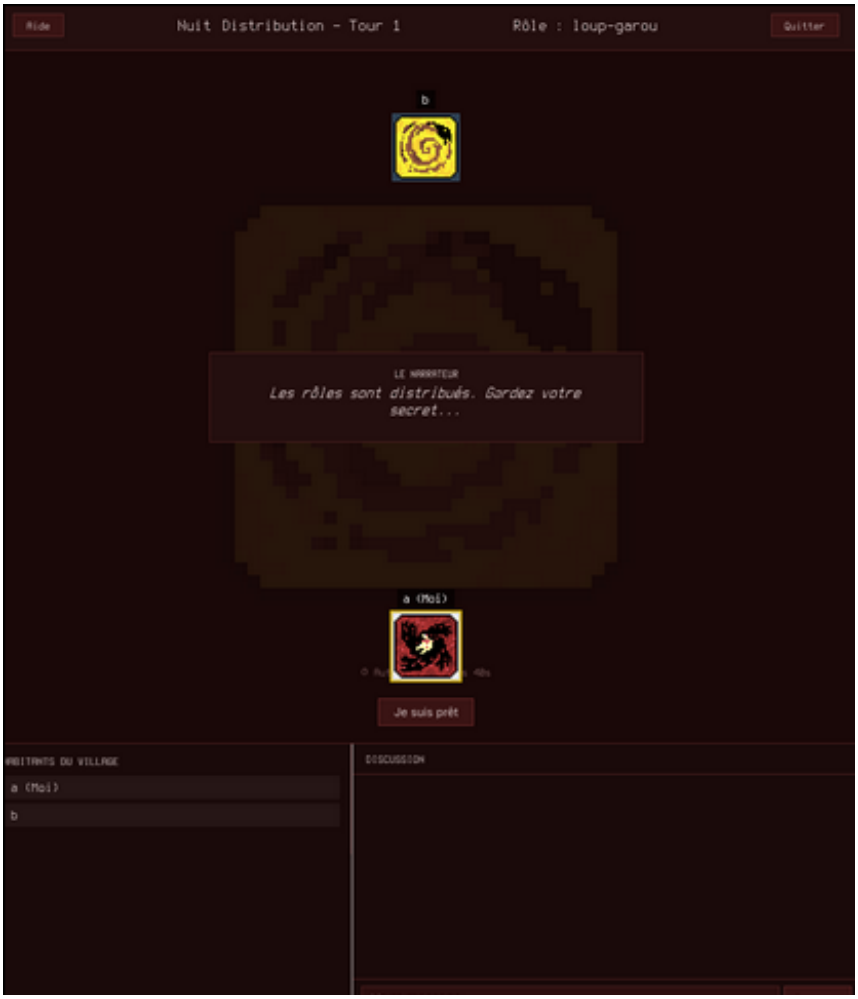

FIGURE 1.5 – Interface de jeu — réception des informations en temps réel

Chapitre 2

Design CSS global, gestion Git et diagrammes

Contribution de ***COULIBALY Bourama***

2.1 Charte graphique et thème visuel

Bourama a conçu et développé l'ensemble des feuilles de style CSS de l'application, assurant une cohérence visuelle sur toutes les pages.

Fichiers CSS produits :

- `css/style.css` — Base partagée
- `css/index.css` — Page lobby
- `css/login.css` — Page de connexion
- `css/register.css` — Page d'inscription
- `css/lobby.css` — Styles du salon d'attente
- `css/game.css` — Interface de jeu
- `css/help.css` — Page d'aide

Palette de couleurs :

Rôle	**Code hex**	parchment !30**Aperçu**
Fond principal	`#f0e8d0`	parchment !30beige parchemin
Fond des sections	`#faf5e8`	parchment !30blanc cassé chaud
Bordures	`#c8b890`	parchment !30or vieilli
Texte principal	`#2c2c1e`	parchment !30brun très sombre
Texte secondaire	`#5d5d4a`	parchment !30brun moyen

Table 2.1 – Palette de couleurs de l'application

Typographie : Police principale *Georgia, serif*, évoquant un style médiéval/fantastique adapté au thème du jeu. Titres en majuscules avec `letter-spacing` pour l'effet parchemin.

2.2 Composants CSS notables

Sections / Panneaux : fond `#faf5e8`, bordure `2px solid #c8b890`, padding généreux, séparateurs horizontaux en pointillés.

Barres fixes : background `#e8dfc4`, `box-shadow` légère, liens en majuscules avec soulignement au survol.

Responsive design : breakpoint à 600px — les blocs de cartes passent en colonne, les barres fixes passent en `relative` pour éviter le chevauchement.

FIGURE 2.1 – Page de connexion (style CSS)

FIGURE 2.2 – Page lobby (style CSS)

2.3 Gestion du dépôt Git et diagrammes

Bourama a assuré la gestion du dépôt Git tout au long du projet :

- Initialisation du dépôt et mise en place de la structure des branches.
- Gestion des merges et résolution des conflits entre contributions.
- Maintien d'un historique de commits propre et lisible.

2.3.1 Diagramme de cas d'utilisation

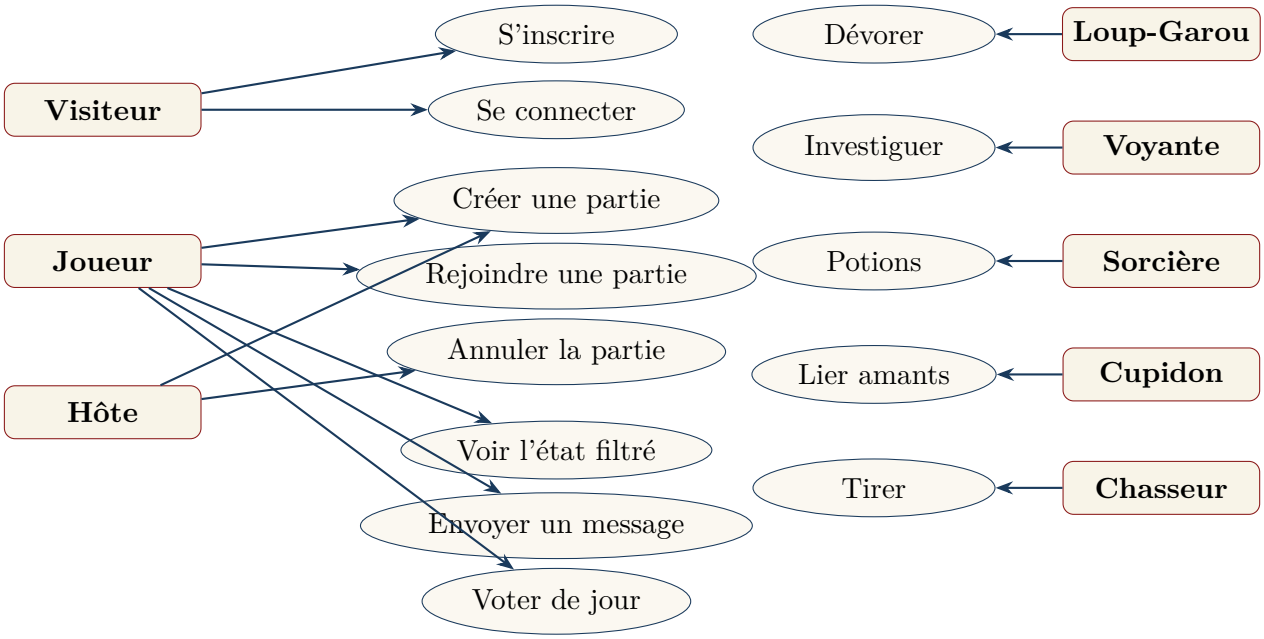

FIGURE 2.3 – Diagramme de cas d'utilisation

Chapitre 3

Page d'aide, mécanique des parties, cartes et pixel art

Contribution de ***CHAUVET Darren***

3.1 Page d'aide

Fichiers : `help.php` / `css/help.css` / `js/help.js`

La page d'aide est accessible uniquement aux joueurs connectés (session vérifiée en haut de `help.php`). Elle se compose de deux sections.

Section 1 — « Comment ça marche ? »

- Créer / rejoindre une partie (code 6 lettres ou liste publique).
- Distribution secrète des rôles au lancement.
- Phase de nuit : les rôles spéciaux agissent dans l'ordre de priorité (voir §3.3).
- Phase de jour : annonce des morts, discussion via le chat, vote pour éliminer un suspect.
- Conditions de victoire (voir §3.5).
- Système d'auto-avance avec compte à rebours de 60 secondes.

Section 2 — « Les cartes » : Présentation interactive de chaque rôle avec animation flip 3D (voir §3.2).

FIGURE 3.1 – Page d'aide — « Comment ça marche ? »

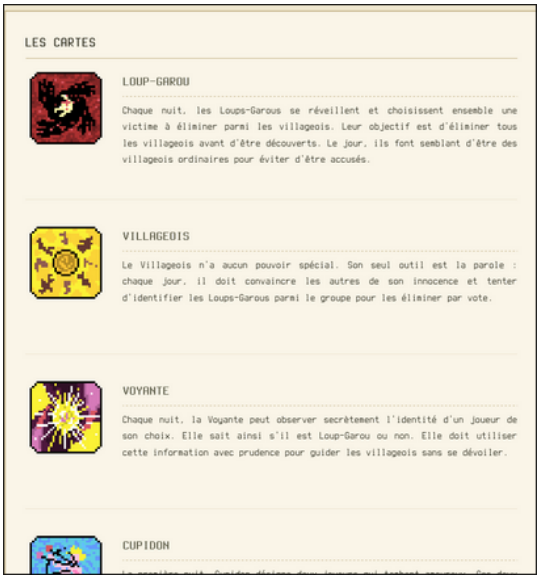

FIGURE 3.2 – Page d'aide — Section « Les cartes »

3.2 Design pixel art des cartes

Dossier : `sprite/`

Chaque carte a été dessinée en pixel art avec **Aseprite** et exportée en PNG. Les sprites sont affichés avec `image-rendering: pixelated` pour conserver le rendu net sans interpolation.

Cartes disponibles :

- `backCard.png` — Dos de carte (commun à tous les rôles)
- `loupGarou.png`, `villageois.png`, `voyante.png`, `cupidon.png`, `chasseur.png`, `sorciere.png`, `petite-fille.png`

Dimensions affichées :

- Page de jeu (`game.html`) : 96 × 96 px
- Page d'aide (`help.php`) : 120 × 170 px (format portrait, perspective 1000px pour le flip)

Animation flip 3D (CSS + JS) :

- `.carte-container` : `perspective` + `cursor: pointer`
- `.carte-inner` : `transform-style: preserve-3d`, `transition 0.6s cubic-bezier(0.4, 0, 0.2, 1)`
- `.carte-front` : face visible par défaut (dos de carte)
- `.carte-back` : `rotateY(180deg)` — révèle l'illustration du rôle
- Au clic (`js/help.js`), la classe `.flipped` est ajoutée/retirée sur `.carte-container`.

FIGURE 3.3 – Cartes en position dos (face cachée)

FIGURE 3.4 – Cartes retournées — rôles révélés

3.3 Priorité d'appel des rôles — Ordre des phases de nuit

Les phases de nuit se succèdent dans un ordre strict, géré côté serveur dans `action.php`.

#	Rôle	Nuit 1	Nuits suivantes
1	Cupidon		—
2	Voyante		
3	Loups-Garous		
4	Sorcière		
5	Chasseur	(si éliminé)	(si éliminé)
6	Petite Fille		

TABLE 3.1 – Ordre de jeu des rôles pendant la nuit

Note : Si un rôle spécial n'est pas activé dans la partie, sa phase est sautée automatiquement.

3.4 Effets et interactions entre les cartes

Loup-Garou

Chaque nuit, tous les loups votent ensemble (`votesLoups`) pour désigner une victime. La victime la plus votée est éliminée. Ils voient les votes de leurs congénères en temps réel.

Villageois

Aucun pouvoir. Participe uniquement au vote de jour. Son seul outil est la parole (chat).

Voyante

Chaque nuit, elle inspecte un joueur de son choix. Le serveur lui renvoie `resultatVoyante` et `cibleVoyante`. Elle doit guider les villageois sans se dévoiler.

Sorcière

Possède deux potions à usage unique :

- `potionVie` — sauve la victime des loups cette nuit.
- `potionMort` — élimine n'importe quel joueur.

Elle voit l'identité de la victime avant de décider. Elle peut utiliser les deux potions la même nuit.

Cupidon

Agit uniquement la première nuit. Désigne deux amants. Si l'un meurt, l'autre meurt de chagrin. Si les deux amants sont de camps opposés, ils forment un 3e camp et gagnent s'ils sont les deux derniers survivants.

Chasseur

Lorsqu'il est éliminé (de nuit ou par vote de jour), `peutTirer` passe à `true`. Il doit immédiatement désigner un joueur à abattre. Son élimination a donc toujours un coût pour ses adversaires.

Petite Fille

Pendant la phase loups, elle peut « espionner » et voir `resultatEspionnage` (liste des loups actifs). Si elle est surprise (`alerteEspionnage = true`), elle est éliminée à la place de la victime initiale. Pouvoir risqué : information précieuse mais mort possible.

Interactions notables :

— **Sorcière + Chasseur** : si la sorcière empoisonne le chasseur, celui-ci peut tout de même tirer avant de mourir.
— **Cupidon + Loup-Garou** : si un loup est lié à un villageois, ils forment un camp indépendant pouvant gagner même si leur camp d'origine perd.

3.5 Conditions de victoire

Camp	Condition de victoire
Villageois	Tous les Loups-Garous sont éliminés
Loups-Garous	Nombre de loups $\geq$ nombre de villageois en vie
Amants	Les deux amants sont les deux derniers survivants

Table 3.2 – Conditions de victoire par camp

Note : En cas d'égalité (loups = villageois), les Loups-Garous l'emportent car ils contrôlent le vote de nuit.

3.6 Système d'auto-avance et compte à rebours

— Chaque phase dispose d'un timestamp de début (`phaseDebutTs`) et d'une durée fixe de 60 secondes (`COOLDOWN_SECONDES` dans `action.php`).
— `etat.php` renvoie ces deux valeurs au client JS.
— Le client affiche un compte à rebours et, à expiration, envoie automatiquement une action `"avancer"` pour passer à la phase suivante.
— Les phases `"attente"` et `"fin"` n'ont pas de cooldown.

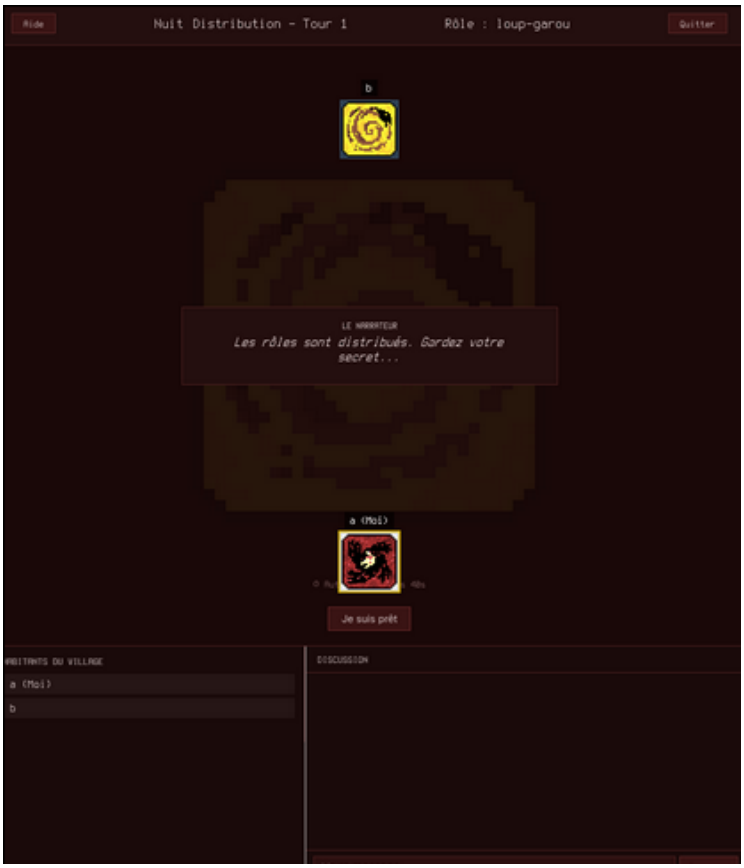

FIGURE 3.5 – Interface de jeu — compte à rebours et état de la partie

Conclusion

Ce projet nous a permis de mettre en pratique plusieurs compétences clés :

- **Gestion de la concurrence** — verrous fichiers (`LOCK_EX` / `LOCK_SH`) pour garantir l'intégrité des données sans base de données.
- **Sécurité web** — jetons CSRF sur toutes les actions POST, hachage des mots de passe avec *bcrypt*, filtrage de l'état par rôle pour prévenir la triche.
- **Architecture client-serveur** — séparation claire entre un frontend stateless (polling 2s) et un backend stateful (JSON + PHP).
- **Design collaboratif** — répartition des tâches en trois domaines complémentaires (backend, CSS/Git, mécanique de jeu).